6. Round – Robin Scheduling

Code:

```c
#include <stdio.h>

int main()
{
    int n, i, tq, time = 0, completed = 0;
    int at[20], bt[20], rem[20], wt[20], tat[20];
    int queue[10000], front = 0, rear = 0;
    int visited[20] = {0}, p;

    printf("Total number of process in the system: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("\nEnter the Arrival and Burst time of Process[%d]\n", i + 1);
        printf("Arrival time is: ");
        scanf("%d", &at[i]);
        printf("Burst time is: ");
        scanf("%d", &bt[i]);
        rem[i] = bt[i];
    }

    printf("Enter the Time Quantum for the process: ");
    scanf("%d", &tq);

    while (completed < n)
    {
        int found = 0;

        for (i = 0; i < n; i++)
        {
```

```
        if (rem[i] > 0 && at[i] <= time && !visited[i])
        {
            queue[rear++] = i;
            visited[i] = 1;
        }
    }

    if (front == rear)
    {
        int next = -1;

        for (i = 0; i < n; i++)
            if (rem[i] > 0 &&
                (next == -1 || at[i] < at[next]))
                next = i;

        if (next != -1)
            time = at[next];

        continue;
    }

    p = queue[front++];
    found = 1;

    if (rem[p] > tq)
    {
        time += tq;
        rem[p] -= tq;
    }
```

```
        {
            time += rem[p];
            rem[p] = 0;
            tat[p] = time - at[p];
            wt[p] = tat[p] - bt[p];
            completed++;
        }

        for (i = 0; i < n; i++)
        {
            if (rem[i] > 0 && at[i] <= time && !visited[i])
            {
                queue[rear++] = i;
                visited[i] = 1;
            }
        }

        if (rem[p] > 0)
            queue[rear++] = p;
    }

    printf("\nProcess No\tBurst Time\tTAT\tWaiting Time\n");

    for (i = 0; i < n; i++)
        printf("Process No[%d]\t%d\t\t%d\t%d\n",
               i + 1, bt[i], tat[i], wt[i]);

    {
        float avgtat = 0, avgwt = 0;

        for (i = 0; i < n; i++)
        {
```

```
        float avgtat = 0, avgwt = 0;

        for (i = 0; i < n; i++)
        {
            avgtat += tat[i];
            avgwt += wt[i];
        }

        printf("Average Turn Around Time: %.2f\n", avgtat / n);
        printf("Average Waiting Time: %.2f\n", avgwt / n);
    }

    return 0;
}
```

Output:

```
Enter the Arrival and Burst time of Process[1]
Arrival time is: 1
Burst time is: 2

Enter the Arrival and Burst time of Process[2]
Arrival time is: 3
Burst time is: 4

Enter the Arrival and Burst time of Process[3]
Arrival time is: 1
Burst time is: 3

Enter the Arrival and Burst time of Process[4]
Arrival time is: 4
Burst time is: 2

Enter the Arrival and Burst time of Process[5]
Arrival time is: 3
Burst time is: 2
Enter the Time Quantum for the process: 4

Process No      Burst Time      TAT     Waiting Time
Process No[1]   2               2       0
Process No[2]   4               7       3
Process No[3]   3               5       2
Process No[4]   2               10      8
Process No[5]   2               9       7
Average Turn Around Time: 6.60
Average Waiting Time: 4.00
```